Aly Muhammad Kazani 24K-0512

DBS Lab 8 Tasks

```
-- 1

CREATE OR REPLACE TRIGGER TRG_EMPLOYEE_AUDIT

AFTER INSERT ON EMPLOYEES

FOR EACH ROW

BEGIN

    INSERT INTO EMPLOYEE_AUDIT(EMPLOYEE_ID, FIRST_NAME, HIRE_DATE,
INSEfRTION_TIMESTAMP)

    VALUES (:NEW.EMPLOYEE_ID, :NEW.fIRST_NAME, :NEW.HIRE_DATE,
SYSTIMESTAMP);

END;


-- 2

CREATE OR REPLACE TRIGGER TRG_STOP_HIGH_SALARY

BEFORE UPDATE OF SALARY ON EMPLOYEES

FOR EACH ROW

BEGIN

    IF :NEW.SALARY > 50000 THEN

        RAISE_APPLICATION_ERROR(-20500, 'Salary cannot be larger than
50000.');

    END IF;

END;


-- 3

CREATE OR REPLACE TRIGGER TRG_DEPT_LOG_NAMES

AFTER UPDATE OF DEPARTMENT_NAME ON DEPARTMENTS

FOR EACH ROW

BEGIN

    INSERT INTO DEPT_LOG(OLD_NAME, NEW_NAME, CHANGE_DATE)

    VALUES(:OLD.DEPARTMENT_NAME, :NEW.DEPARTMENT_NAME, SYSDATE);
```

```
END;

-- 4
CREATE OR REPLACE TRIGGER TRG_LOG_JOB_CHANGE
AFTER UPDATE OF JOB_ID ON EMPLOYEES
FOR EACH ROW
BEGIN
    INSERT INTO JOB_HISTORY(EMPLOYEE_ID, OLD_JOB, NEW_JOB, CHANGE_DATE)
    VALUES(:NEW.EMPLOYEE_ID, :OLD.JOB_ID, :NEW.JOB_ID, SYSDATE);
END;

-- 5
CREATE OR REPLACE TRIGGER TRG_PREVENT_DEPT_DELETE
BEFORE DELETE ON DEPARTMENTS
FOR EACH ROW
DECLARE
    VALS NUMBER;
BEGIN
    SELECT COUNT(*) INTO VALS FROM EMPLOYEES
        WHERE DEPARTMENT_ID = :OLD.DEPARTMENT_ID;
    IF VALS > 0 THEN
        RAISE_APPLICATION_ERROR(-20501, 'Cannot delete depaartment with
existing employees.');
    END IF;
END;

-- 6
CREATE OR REPLACE TRIGGER TRG_UPDATE_DEPT_TOTAL_SAL
AFTER INSERT OR UPDATE OF SALARY, DEPARTMENT_ID ON EMPLOYEES
BEGIN
    UPDATE DEPARTMENTS D
```

```
    SET TOTAL_SALARY = (SELECT SUM(SALARY) FROM EMPLOYEES E
        WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID);
END;

-- 7
CREATE OR REPLACE TRIGGER TRG_EMPLOYEE_TERMINATIONS
BEFORE DELETE ON EMPLOYEES
FOR EACH ROW
BEGIN
    INSERT INTO EMPLOYEE_TERMINATIONS(EMPLOYEE_ID, FIRST_NAME, LAST_NAME,
DEPARTMENT_ID, TERMINATION_DATE)
    VALUES(:OLD.EMPLOYEE_ID, :OLD.FIRST_NAME, :OLD.LAST_NAME,
:OLD.DEPARTMENT_ID, SYSDATE);
END;

-- 8
CREATE OR REPLACE TRIGGER TRG_CHECK_HIRE_DATE
BEFORE INSERT ON EMPLOYEES
FOR EACH ROW
BEGIN
    IF :NEW.HIRE_DATE < TO_DATE('1990-01-01', 'YYYY-MM-DD') THEN
        RAISE_APPLICATION_ERROR(-20502, 'Hire date must be on or after
January 1, 1990.');
    END IF;
END;

-- 9
CREATE OR REPLACE TRIGGER TRG_SALARY_HISTORY
AFTER UPDATE OF SALARY ON EMPLOYEES
FOR EACH ROW
BEGIN
```

```
    IF :OLD.SALARY != :NEW.SALARY THEN
        INSERT INTO SALARY_HISTORY(EMPLOYEE_ID, OLD_SALARY, NEW_SALARY,
UPDATE_DATE)
        VALUES(:NEW.EMPLOYEE_ID, :OLD.SALARY, :NEW.SALARY, SYSDATE);
    END IF;
END;

-- 10
CREATE OR REPLACE TRIGGER TRG_SET_MANAGER_LEVEL
BEFORE INSERT ON EMPLOYEES
FOR EACH ROW
DECLARE
    MGR_JOB_ID VARCHAR2(20);
BEGIN
    SELECT JOB_ID INTO MGR_JOB_ID FROM EMPLOYEES WHERE EMPLOYEE_ID =
:NEW.MANAGER_ID;
    :NEW.MANAGER_LEVEL := MGR_JOB_ID;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        :NEW.MANAGER_LEVEL := NULL;
END;

-- 11
CREATE OR REPLACE TRIGGER TRG_DEPT_BUDGET_MAX
BEFORE INSERT OR UPDATE OF SALARY ON EMPLOYEES
FOR EACH ROW
DECLARE
    CURRENT_TOTAL NUMBER;
BEGIN
    SELECT SUM(SALARY) INTO CURRENT_TOTAL FROM EMPLOYEES
```

```
    WHERE DEPARTMENT_ID = :NEW.DEPARTMENT_ID AND EMPLOYEE_ID !=
NVL(:NEW.EMPLOYEE_ID, -1);
    IF (CURRENT_TOTAL + :NEW.SALARY) > 200000 THEN
        RAISE_APPLICATION_ERROR(-20503, 'Total department salary cannot be
greatre than 200,000.');
    END IF;
END;


-- 12
CREATE OR REPLACE TRIGGER TRG_PROMOTION_BONUS
BEFORE UPDATE OF JOB_ID ON EMPLOYEES
FOR EACH ROW
DECLARE
    OLD_RANK NUMBER;
    NEW_RANK NUMBER;
BEGIN
    SELECT RANK INTO OLD_RANK FROM JOB_HIERARCHY WHERE JOB_ID =
:OLD.JOB_ID;
    SELECT RANK INTO NEW_RANK FROM JOB_HIERARCHY WHERE JOB_ID =
:NEW.JOB_ID;
    IF NEW_RANK > OLD_RANK THEN
        :NEW.SALARY := :OLD.SALARY * 1.05;
    END IF;
END;


-- 13
CREATE OR REPLACE TRIGGER TRG_ARCHIVE_DEPT_EMPLOYEES
BEFORE DELETE ON DEPARTMENTS
FOR EACH ROW
BEGIN
    INSERT INTO EMPLOYEE_ARCHIVE(SELECT * FROM EMPLOYEES
```

```
                                    WHERE DEPARTMENT_ID =
:OLD.DEPARTMENT_ID);
    DELETE FROM EMPLOYEES
        WHERE DEPARTMENT_ID = :OLD.DEPARTMENT_ID;
END;

-- 14
CREATE OR REPLACE TRIGGER TRG_SYNC_MGR_DEPT
BEFORE UPDATE OF MANAGER_ID ON EMPLOYEES
FOR EACH ROW
DECLARE
    MGR_DEPT_ID NUMBER;
BEGIN
    SELECT DEPARTMENT_ID INTO MGR_DEPT_ID FROM EMPLOYEES
        WHERE EMPLOYEE_ID = :NEW.MANAGER_ID;
    :NEW.DEPARTMENT_ID := MGR_DEPT_ID;
END;

-- 15
CREATE OR REPLACE TRIGGER TRG_NO_DUPLICATE_NAMES
BEFORE INSERT ON EMPLOYEES
FOR EACH ROW
DECLARE
    COUNT NUMBER;
BEGIN
    SELECT COUNT(*) INTO COUNT FROM EMPLOYEES
    WHERE UPPER(FIRST_NAME) = UPPER(:NEW.FIRST_NAME)
    AND UPPER(LAST_NAME) = UPPER(:NEW.LAST_NAME);
    IF COUNT > 0 THEN
        RAISE_APPLICATION_ERROR(-20504, 'Employee, with the same name,
already exists.');
```

```
    END IF;
END;
```